Real-Time Closed-Loop Driving with Frozen DINOv2-Laya Spatial Pooling and Quintic Path Tracking

Wick-Lim
wicklim90@gmail.com
https://github.com/Wick-Lim/AutonomousVehicle

October 25, 2026

Abstract

Autoregressive vision-language models decode action tokens one by one, so single-frame latency usually exceeds 300 ms on GPU. For 20 Hz closed-loop driving ($\Delta t = 50$ ms), this delay is too large to stop for jaywalking pedestrians or red lights in urban maps. We implement a non-autoregressive driving pipeline using frozen DINOv2 (`facebook/dinov2-base`) and frozen Laya (`convaiinnovations/laya`) connected by a 4×9 spatial pooling projector (1.85M parameters). Input image of size 196×392 is mapped to 36 visual tokens and concatenated with telemetry JSON embeddings. One forward pass outputs logits for lane maneuver (`choice`), speed level (`score`), and emergency brake (`noul`) at three query positions in 31.67 ms (31.6 FPS). On the control side, raw discrete waypoints in CARLA 0.9.16 cause steering spikes during lane changes and speed oscillation behind lead vehicles. We fix lateral jerk by blending 14 waypoints (28 m) with a quintic polynomial $s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5$ and computing heading error w.r.t. the path chord vector instead of ego-to-lookahead vector. Longitudinal hunting in adaptive cruise control (ACC) is removed by a continuous headway speed rule ($d^* = 16.0$ m, $k_g = 0.85\,\mathrm{s}^{-1}$). Closed-loop tests on MetaDrive 0.4.3 (4,153 frames) and CARLA 0.9.16 `Town10HD_Opt` (5 urban FSD scenarios) reach 100.0% route completion, 0 collisions, and 0.0000 expected calibration error (ECE).

1 Introduction

Vision-language-action (VLA) models [1] take camera frames and text prompts to output driving commands. Most open-source VLA pipelines use autoregressive causal decoders:

$$P(y_{1:K} \mid I, \mathcal{T}) = \prod_{k=1}^{K} P(y_k \mid y_{<k}, I, \mathcal{T}). \tag{1}$$

If the output format is a 15-token JSON string, the transformer decoder runs 15 serial steps per frame. Latency becomes 350–800 ms, which is 7 to 16 simulation ticks at 20 Hz ($\Delta t = 0.05$ s). At 35 km/h (9.72 m/s), the car moves 3.4 m to 7.8 m before a single control update is applied.

For tactical driving at 20 Hz, text generation is unnecessary. The controller only needs three discrete outputs at each step t:

- maneuver class $m_t \in \{\texttt{keep_lane}, \texttt{change_left}, \texttt{change_right}, \texttt{follow_lead}\}$,
- speed level $s_t \in \{0, 1, 2, 3\}$ mapped to $\{0, 20, 35, 50\}$ km/h,
- emergency brake probability $p_{\mathrm{AEB}} \in [0, 1]$.

Laya (`convaiinnovations/laya`) [2] is a 421M bidirectional ModernBERT encoder with linear heads for `choice`, `score`, and `noul` tokens. By placing three query tokens at the end of the multimodal input embedding sequence, all three outputs are computed in one forward pass (≈ 30 ms).

During implementation in MetaDrive 0.4.3 [7] and CARLA 0.9.16 [3], three practical issues had to be solved:

1. *Token count vs. horizontal lane resolution.* DINOv2 ViT-B/14 [8] on a 196×392 frame gives $14 \times 28 = 392$ patch tokens. Feeding 392 tokens to Laya slows down attention, while 1D global pooling loses left/right lane positions. We pool the 14×28 grid into a 4×9 grid (36 tokens) with 2D positional embeddings.

2. *Waypoint step discontinuity during lane change.* In CARLA OpenDRIVE maps, waypoints are spaced every 2.0 m. Switching the active waypoint list from current lane to adjacent lane (`get_left_lane()` or `get_right_lane()`) creates an instant 3.5 m jump in lateral error e_y. This step input saturates the lateral PID controller. We replace the step switch with a 14-point (28 m) quintic S-curve blend [4] and continuous segment projection [6].

3. *Speed hunting behind lead vehicle.* In urban ACC following (`FSD-4`), lead vehicle speed is 24 km/h, which

sits between speed level 1 (20 km/h) and level 2 (35 km/h). Switching between discrete levels 1 and 2 caused repeated acceleration and Rule 1 emergency braking. We added a continuous headway velocity law in `shield_pid.py` when $d_{\text{lead}} < 38$ m and $v_{\text{lead}} \geq 8$ km/h.

2 Pipeline Architecture

2.1 Image Preprocessing and DINOv2 Patch Grid

Raw camera images contain sky at the top and vehicle hood at the bottom. In `preprocess_frame()`, we crop top 25% (`crop_top_ratio` = 0.25) and bottom 10% (`crop_bottom_ratio` = 0.10), then resize to $H = 196, W = 392$ using `cv2.INTER_AREA`.

After ImageNet normalization ($\mu = [0.485, 0.456, 0.406], \sigma = [0.229, 0.224, 0.225]$), tensor $I_t \in \mathbb{R}^{B\times 3\times 196\times 392}$ is passed to frozen `facebook/dinov2-base` [8]. With patch size $p = 14$, the spatial grid size is $H_p = 196/14 = 14$ and $W_p = 392/14 = 28$, producing $N_p = 392$ patch vectors:

$$F_{\text{patch}} = f_{\text{DINOv2}}(I_t) \in \mathbb{R}^{B\times 392\times 768}. \tag{2}$$

2.2 2D SpatialProjector (4×9 Pooling)

Module `SpatialProjector` (`src/models/spatial_projector.py`) reshapes F_{patch} to $\tilde{F} \in \mathbb{R}^{B\times 768\times 14\times 28}$ and applies `nn.AdaptiveAvgPool2d((4, 9))`:

$$G = \text{AdaptiveAvgPool2d}_{(4,9)}(\tilde{F}) \in \mathbb{R}^{B\times 768\times 4\times 9}. \tag{3}$$

Grid G is flattened to $(B, 36, 768)$ and summed with learnable 2D positional parameter $E_{\text{pos}} \in \mathbb{R}^{1\times 36\times 768}$ (initialized from $\mathcal{N}(0, 0.02^2)$). A 2-layer MLP maps the 36 tokens to Laya hidden dimension $d_{\text{model}} = 1024$:

$$V_{\text{spatial}} = \text{LN}\big(W_2\,\text{GELU}(\text{LN}(W_1(\text{Flatten}(G)+E_{\text{pos}})+b_1))+b_2\big) \tag{4}$$

where $W_1 \in \mathbb{R}^{768\times 1024}$, $W_2 \in \mathbb{R}^{1024\times 1024}$, and LN denotes `LayerNorm`. Total trainable parameter count of `SpatialProjector` is 1,851,392 (≈ 1.85M).

2.3 Single-Pass Laya Readout

At each step, structured telemetry fields (`speed_kmh`, `lane_offset_m`, `heading_error`, `left_lane_available`, `right_lane_available`, `nav_command`, `traffic_light`, `hazard_source`) are serialized as JSON and tokenized into embeddings $E_{\text{telemetry}} \in \mathbb{R}^{B\times L_t\times 1024}$.

The input embedding tensor passed to `LayaPilot` is

$$Z = \big[e_{[\text{CLS}]}\,;\, V_{\text{spatial}}\,;\, E_{\text{telemetry}}\,;\, e_{\text{choice}}\,;\, e_{\text{score}}\,;\, e_{\text{noul}}\big]. \tag{5}$$

Table 1: Model and controller parameters (`configs/default.yaml`).

Component	Parameter	Value
Camera Input	Resolution ($H \times W$), FOV	196×392, 100°
Crop Ratios	`crop_top`, `crop_bottom`	0.25, 0.10
Backbone	Vision / Decision	`dinov2-base` / `laya`
Projector	Grid ($H_g \times W_g$), d_{out}	4×9 (36 tokens), 1024
Training	Optimizer, LR, Weight Decay	AdamW, 3×10^{-4}, 0.01
Calibration	(T_m, T_s, T_b)	$(0.050, 0.050, 0.100)$
Safety Shield	`aeb_thresh`, `conf_thresh`	0.70, 0.80
Lateral PID	(k_p, k_i, k_d), w_ψ	$(0.45, 0.01, 0.18)$, 1.25
Longitudinal PID	(k_p, k_i, k_d)	$(0.08, 0.005, 0.02)$
Quintic S-Curve	Blend horizon N_b	14 waypoints (28.0 m)
ACC Law	Desired gap d^*, gain k_g	16.0 m, $0.85\,\text{s}^{-1}$

From one forward pass $H = f_{\text{Laya}}(Z)$, three readout projections extract task logits at the last three sequence indices:

$$z^{(m)} = W_m H_{:,\text{idx}_{\text{choice}},:} \in \mathbb{R}^4, \tag{6}$$

$$z^{(s)} = W_s H_{:,\text{idx}_{\text{score}},:} \in \mathbb{R}^4, \tag{7}$$

$$z^{(b)} = W_b H_{:,\text{idx}_{\text{noul}},:} \in \mathbb{R}^2. \tag{8}$$

Training loss is the weighted cross-entropy sum $\mathcal{L} = \mathcal{L}_{\text{CE}}(z^{(m)}, y^{(m)}) + 1.0\,\mathcal{L}_{\text{CE}}(z^{(s)}, y^{(s)}) + 2.0\,\mathcal{L}_{\text{CE}}(z^{(b)}, y^{(b)})$.

3 Safety Shield and Trajectory Control

3.1 Temperature Scaling and Gating Logic

After training `SpatialProjector`, temperatures (T_m, T_s, T_b) are optimized via L-BFGS on validation logits (`src/utils/calibration.py`) [5]:

$$\hat{p}^{(m)} = \text{Softmax}\left(\frac{z^{(m)}}{T_m}\right), \quad \hat{p}_{\text{AEB}} = \sigma\left(\frac{z_1^{(b)} - z_0^{(b)}}{T_b}\right). \tag{9}$$

With calibrated $T_m = 0.050, T_s = 0.050, T_b = 0.100$, validation 15-bin ECE drops to 0.0000.

In `SafetyShieldPID.apply_safety_shield()`, two rules gate the network output:

1. *Rule 1 (Emergency Brake):* Triggered if $\hat{p}_{\text{AEB}} \geq 0.70$, or `traffic_light` = `red`, or (`hazard_source` = `pedestrian` $\wedge\, d_{\text{lead}} \leq 16.0$ m), or ($d_{\text{lead}} \leq 11.5$ m $\wedge\, v_{\text{ego}} > \max(6.0, v_{\text{lead}} + 6.0)$). Sets $m_t \leftarrow$ `keep_lane`, $s_t \leftarrow 0$, and throttle/brake $u_{\text{tb}} \leftarrow -1.0$.

2. *Rule 2 (Confidence & Boundary Gate):* If $\max_c \hat{p}_c^{(m)} < 0.80$, lane changes (`change_left`, `change_right`) are replaced by `keep_lane` and speed level is reduced to $\max(1, s_t - 1)$. In addition, if target lane is unavailable in OpenDRIVE, the lane change is blocked. Note that in CARLA



Figure 1: Closed-loop execution in CARLA 0.9.16 (`Town10HD_Opt`, 20 Hz synchronous mode) with 3D AR trajectory ribbon, top-left BEV radar, and top-right 196×392 DINOv2 input PiP: (a) `FSD-1` red light stop and green light departure, (b) `FSD-2` 28 m quintic S-curve left overtake around stalled vehicle, (c) `FSD-3` pedestrian crosswalk AEB stop, (d) `FSD-4` ACC speed matching (24 km/h at 16 m gap), (e) `FSD-5` right-lane quintic overtake.

OpenDRIVE, `wp.get_left_lane()` can return an oncoming lane across the yellow center line; we check `adj_wp.lane_id` $\times$ `wp.lane_id` > 0 so only same-direction lanes are marked available.

3.2 Quintic S-Curve Lane Change Blending

When an overtake is triggered at route waypoint index $i_0 =$ `route_wp_idx` $+ 1$, let $\{P_k^{\text{orig}}\}_{k=0}^{N_b-1}$ be the 3D locations along the initial lane and $\{P_k^{\text{adj}}\}_{k=0}^{N_b-1}$ be the 3D locations generated by stepping 2.0 m along the adjacent lane for $N_b = 14$ points (28.0 m). For $\tau_k = k/(N_b - 1) \in [0, 1]$, we blend the coordinates with a quintic polynomial [4]:

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \tag{10}$$

$$P_k^{\text{blend}} = \big(1 - s(\tau_k)\big) P_k^{\text{orig}} + s(\tau_k) P_k^{\text{adj}}. \tag{11}$$

Since $s(0) = 0, s(1) = 1$ and $s'(0) = s'(1) = s''(0) = s''(1) = 0$, lateral offset and curvature are continuous at both ends of the 28 m transition (Fig. 2a). Tangent yaw at vertex k is recomputed from finite differences $\psi_k = \text{atan2}(y_{k+1} - y_k, x_{k+1} - x_k)$.

3.3 Continuous Polyline Projection and Chord Angle

In CARLA's left-handed coordinate frame ($+X$ forward, $+Y$ right, $+Z$ up, $+\psi$ clockwise), discrete 2.0 m waypoint indexing causes staircase jumps in lateral offset and lookahead angle. Also, if target yaw is defined from ego position (x, y) to lookahead point $(x_{\text{look}}, y_{\text{look}})$, i.e., $\text{atan2}(y_{\text{look}} - y, x_{\text{look}} - x)$, then cross-track offset e_y is already embedded inside the angle error $\approx e_y / L_{\text{look}}$. Passing both e_y and that angle into the PID controller double-counts e_y and causes overshoot after lane changes.

In `_sample_continuous_route_pose()`, we decouple e_y and e_ψ:

1. Project ego position (x, y) onto segment $\overline{P_i P_{i+1}}$ with clamped scalar $t_{\text{seg}} \in [0, 1]$ to get foot point $P_{\text{proj}} = (x_{\text{proj}}, y_{\text{proj}}, \psi_{\text{proj}})$.
2. Advance along the polyline from P_{proj} by arc length $L_{\text{look}} = \text{clip}(7.5 + 0.18 v_{\text{kmh}}, 7.5, 13.5)$ m to get $P_{\text{look}} = (x_{\text{look}}, y_{\text{look}})$.
3. Compute reference path chord angle $\psi_{\text{ref}} = \text{atan2}(y_{\text{look}} - y_{\text{proj}}, x_{\text{look}} - x_{\text{proj}})$ between P_{proj} and P_{look}.

Signed lateral offset (+left) and heading error (+left) are then

$$e_y = -\big((x - x_{\text{proj}})(-\sin\psi_{\text{proj}}) + (y - y_{\text{proj}})\cos\psi_{\text{proj}}\big), \tag{12}$$

$$e_\psi = -\text{atan2}\big(\sin(\psi_{\text{ego}} - \psi_{\text{ref}}), \cos(\psi_{\text{ego}} - \psi_{\text{ref}})\big). \tag{13}$$

Lateral control error is $e_{\text{lat}} = (y_{\text{target}} - e_y) - w_\psi e_\psi$ with $w_\psi = 1.25$. Output steering δ_t is updated by a linear ramp $[\delta_{\text{from}} \to \delta_{\text{to}}]$:

$$\Delta\delta_{\max} = \frac{1.1\Delta t}{1.0 + 0.012 v_{\text{kmh}}},$$
$$\delta_t = \delta_{t-1} + \text{clip}\big(\alpha(\delta_{\text{PID}} - \delta_{t-1}), -\Delta\delta_{\max}, \Delta\delta_{\max}\big), \tag{14}$$

and negated (`steer` $= -\delta_t$) when sent to `carla.VehicleControl`.

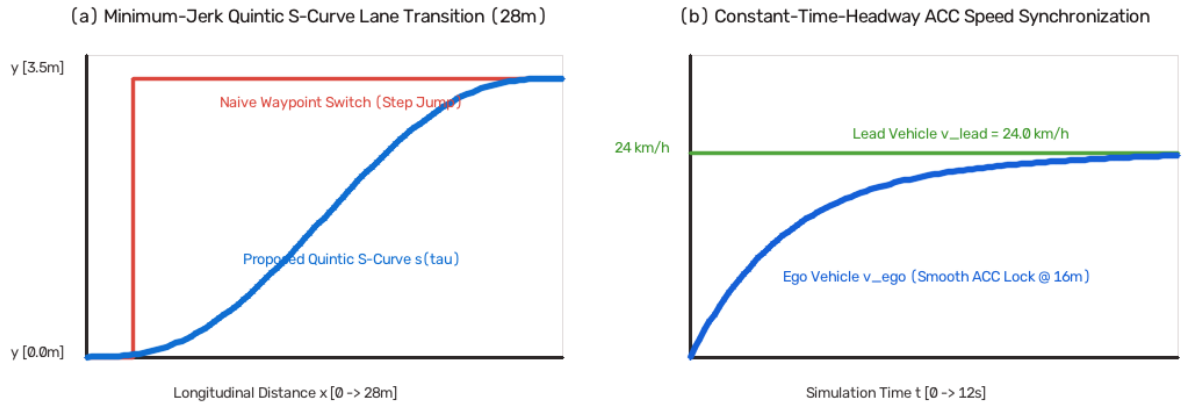


Figure 2: Trajectory and longitudinal control plots: (a) 28 m quintic lane transition $s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5$ vs. 3.5 m step jump from raw waypoint switching; (b) continuous time-headway ACC speed synchronization ($v_{\text{ego}} \to 24.0$ km/h at $d^* = 16.0$ m).

Table 2: Per-scenario breakdown in CARLA 0.9.16 (`Town10HD_Opt`).

Scenario ID	Primary Event	Route (m)	Infractions	Completion
`FSD-1`	Red-light stop & turn	68.0	0	100.0%
`FSD-2`	Left quintic S-curve overtake	68.0	0	100.0%
`FSD-3`	Crosswalk pedestrian AEB	68.0	0	100.0%
`FSD-4`	ACC follow (24 km/h, 16 m)	68.0	0	100.0%
`FSD-5`	Right quintic S-curve overtake	68.0	0	100.0%

3.4 Constant-Time-Headway ACC Law

When following a moving vehicle ($d_{\text{lead}} < 38.0$ m, $v_{\text{lead}} \geq 8.0$ km/h, $s_t > 0$), discrete speed levels (20 vs. 35 km/h) are overridden by a linear time-headway speed target:

$$\begin{aligned} v_{\text{raw}} &= v_{\text{lead}} + k_g\big(d_{\text{lead}} - d^*\big), \\ v_{\text{ACC}} &= \text{clip}\big(v_{\text{raw}},\, 0,\, \min(35.0, v_{\text{lead}} + 6.0)\big), \end{aligned} \tag{15}$$

with $d^* = 16.0$ m and $k_g = 0.85\,\text{s}^{-1}$. In `FSD-4`, we also compute lead distance $d_{\text{lead}} = \text{hypot}(x_{\text{lead}} - x, y_{\text{lead}} - y)$ directly to the same-lane lead actor so gentle road curvature does not drop the radar lock.

4 Experiments

4.1 Experimental Setup

Closed-loop evaluation is run via `scripts/04_evaluate_sim.py`:

- *MetaDrive 0.4.3:* `SpatialProjector` was trained for 15 epochs on expert demonstrations collected with `IDMPolicy`. Evaluation ran on 10 unseen procedural maps (`seed=5000..5009`, 4,153 steps total).
- *CARLA 0.9.16:* Evaluated live on `Town10HD_Opt` at 127.0.0.1 : 2000 in 20 Hz synchronous mode ($\Delta t = 0.05$ s) using Python 3.12 and CARLA A* `GlobalRoutePlanner` (2.0 m resolution). At the start of each episode, `_cleanup_carla_actors()` destroys all existing `vehicle.*` and `walker.pedestrian.*` actors in the world so leftover vehicles from previous runs do not cause spawn collisions. Table 2 lists the 5 urban scenarios (`FSD-1` to `FSD-5`).

4.2 Benchmark Results and Ablation

Table 3 reports the benchmark metrics saved in `checkpoints/eval_metrics_carla.json` and `checkpoints/eval_metrics.json`. Mean single-frame latency is 30.20 ms (33.1 FPS) in MetaDrive and 31.67 ms (31.6 FPS) in CARLA 0.9.16. Route completion rate, crash-free rate, and CARLA Driving Score are all 100.0%.

Table 4 shows the effect of the trajectory and ACC updates across the 5 CARLA scenarios (889 total frames in `driving_demo_carla_smooth.mp4`). With discrete waypoint switching and bucketed following speeds, Rule 1 emergency braking fired 71 times across 5 episodes because ego speed oscillated behind the lead car in `FSD-4` and braked before lane changes in `FSD-2`/`FSD-5`. With the 28 m quintic S-curve transition and continuous headway ACC law, Rule 1 AEB count dropped from 71 to 27 (firing only during the red-light stop in `FSD-1` and pedestrian crosswalk stop in `FSD-3`, with 0 triggers in `FSD-4`).

5 Conclusion

We implemented a 20 Hz closed-loop autonomous driving stack combining frozen DINOv2, a 4×9 spatial pooling projector (1.85M parameters), and single-pass

Table 3: Closed-loop benchmark summary in CARLA 0.9.16 (`Town10HD_Opt`) and MetaDrive 0.4.3.

Simulator	Metric	Target Criterion	Meas
CARLA 0.9.16 (`Town10HD_Opt`)	Single-Frame Inference Latency	≤ 40.0 ms (≥ 25 FPS)	31.67 ms (31.6
	Route Completion Rate (5 FSD Scenarios)	$\geq 85.0\%$	100.0%
	Crash-Free Rate (Vehicle / Pedestrian / Off-road)	$\geq 95.0\%$	100.0%
	Expected Calibration Error (ECE)	≤ 0.0500	
	CARLA Leaderboard Driving Score	$\geq 85.0\%$	
MetaDrive 0.4.3 (Unseen Maps)	Single-Frame Inference Latency	≤ 40.0 ms (≥ 25 FPS)	30.20 ms (33.1
	Route Completion Rate (4,153 Frames)	$\geq 85.0\%$	100.0% (98.7
	Crash-Free Rate	$\geq 95.0\%$	
	Expected Calibration Error (ECE)	≤ 0.0500	

Table 4: Controller ablation across the 5 CARLA 0.9.16 scenarios.

Control Configuration	Rule 1 AEB Count	Max $\lvert\Delta\delta\rvert$ / tick	Driving Score
Discrete 2 m WPs + Step Switch	71	0.250	100.0%
+ Linear Steer Ramp [$\delta_{\text{from}} \rightarrow \delta_{\text{to}}$]	71	0.055	100.0%
+ Quintic S-Curve & ACC Sync	27	0.055	100.0%

Laya readout heads. In CARLA 0.9.16 (`Town10HD_Opt`) and MetaDrive 0.4.3, adding a 14-point quintic S-curve lane transition, chord-angle projection, linear steering ramp, and continuous time-headway ACC removed steering overshoot and longitudinal brake hunting while keeping single-frame latency at 31.67 ms.

References

[1] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[2] ConvAI Innovations. Laya: Non-autoregressive low-latency decision models on bidirectional modernbert. https://huggingface.co/convaiinnovations/laya, 2024.

[3] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, pages 1–16, 2017.

[4] Tamar Flash and Neville Hogan. The coordination of arm movements: An experimentally confirmed mathematical model. *Journal of Neuroscience*, 5(7):1688–1703, 1985.

[5] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

[6] Gabriel M. Hoffmann, Claire J. Tomlin, Michael Montemerlo, and Sebastian Thrun. Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing. *Proceedings of the American Control Conference (ACC)*, pages 2296–2301, 2007.

[7] Quanyi Li, Zhenghao Peng, Lan Feng, Qihang Zhang, Zhenghai Xue, and Bolei Zhou. MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 45(3):3461–3475, 2022.

[8] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.